Lecture 1

Introduction to Deep Learning for Language Modeling

Lecture Notes

MAI554 – Deep Learning for NLP

Prof. Anis Koubaa

College of Engineering, Alfaisal University

akoubaa@alfaisal.edu

Spring 2025

How to Use These Notes

These notes distill the key ideas from Lecture 1 into a logical flow. Each section follows a pattern: **intuition first, then math, then a concrete example**. Self-check questions appear throughout the text. Full solutions are collected in the Appendix.

Contents

1 From AI to Deep Learning **2**
1.1 The AI Hierarchy . . . 2
1.2 What Makes Deep Learning "Deep"? . . . 2

2 Traditional Programming vs. Machine Learning **2**
2.1 Two Paradigms . . . 3
2.2 When to Use Which? . . . 3

3 Data Categories **3**
3.1 Structured vs. Unstructured Data . . . 3

4 Learning Paradigms **4**
4.1 Supervised Learning . . . 4
4.2 Unsupervised Learning . . . 4
4.3 Reinforcement Learning . . . 4

5 Machine Learning vs. Deep Learning **4**
5.1 Six Key Differences . . . 5
5.2 The Feature Extraction Gap . . . 5

6 Computer Vision Foundations **5**
6.1 How Computers See Images . . . 5
6.2 From Pixels to Features . . . 6

7 The ImageNet Challenge **6**
7.1 What is ImageNet? . . . 6
7.2 The Deep Learning Revolution . . . 6

8 Convolutional Neural Networks (CNNs) **7**
8.1 Core Idea . . . 7
8.2 Building Blocks of a CNN . . . 7
8.3 Feature Learning Hierarchy . . . 8

9 Deep Learning Evolution: 2010–2015 **8**
9.1 From Handcrafted Features to Learned Representations . . . 8

10 Computer Vision Tasks **9**

11 Generative Adversarial Networks (GANs) 9
11.1 The Core Idea . . . 9
11.2 The GAN Loss Functions . . . 10
11.3 Training Process . . . 10
11.4 Applications . . . 10

12 NLP Evolution Timeline 11

13 NLP Models: RNN, LSTM, GRU 11
13.1 Recurrent Neural Networks (RNN) . . . 11
13.2 Long Short-Term Memory (LSTM) . . . 12
13.3 Gated Recurrent Unit (GRU) . . . 12

14 NLP Applications 12

15 Self-Attention and Transformers Preview 13
15.1 The Limitation of RNNs . . . 13
15.2 The Self-Attention Innovation . . . 13
15.3 Transformer Architecture (Preview) . . . 13

16 Sequential Data and Model Architectures 14
16.1 Types of Sequential Data . . . 14
16.2 Sequence-to-Sequence Architectures . . . 14

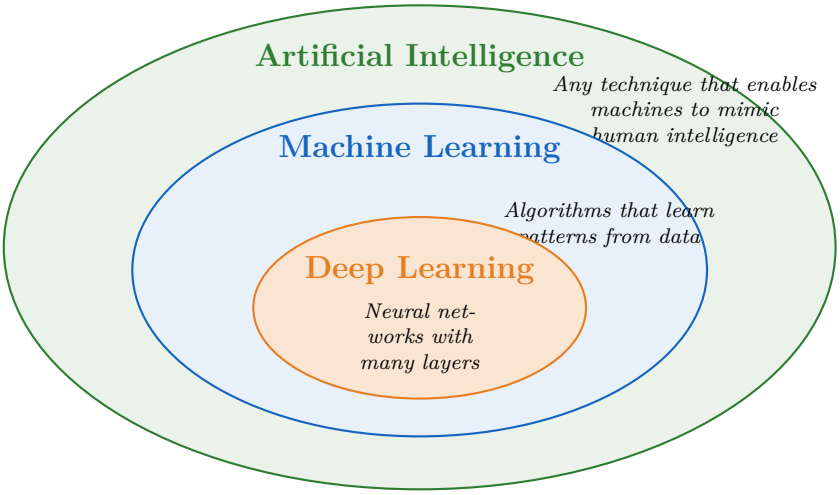
17 Origin of Recurrent Neural Networks 14
17.1 Elman 1990: “Finding Structure in Time” . . . 14

Appendix: Solutions to Self-Check Questions 16

From AI to Deep Learning

The AI Hierarchy

Artificial Intelligence, Machine Learning, and Deep Learning are nested fields, each a subset of the one above:



Key Insight

AI is the broadest goal: making machines behave intelligently. **ML** is one approach: letting machines learn from data instead of being explicitly programmed. **DL** is one type of ML: using deep neural networks (many layers) that can automatically learn hierarchical features from raw data.

What Makes Deep Learning "Deep"?

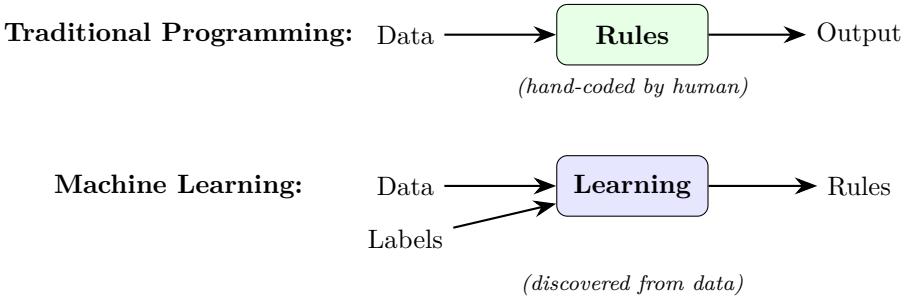
A neural network is "deep" when it has **multiple hidden layers** (typically more than two). Each layer learns increasingly abstract representations:

- **Layer 1**: raw signals (edges, pixel intensities)
- **Layer 2**: simple patterns (textures, curves)
- **Layer 3+**: complex concepts (eyes, wheels, sentences)

The depth enables the network to compose simple features into complex, high-level representations—this is the power of deep learning.

Traditional Programming vs. Machine Learning

Two Paradigms



Intuition

Traditional programming: a human writes explicit rules ("if temperature > 30, turn on AC").
Machine learning: the computer discovers the rules by looking at examples (data + labels). This is essential when the rules are too complex to write by hand—such as recognizing faces, translating languages, or understanding speech.

When to Use Which?

Criterion	Traditional	Machine Learning
Rules are known and simple	✓	
Rules are complex or unknown		✓
Large labeled data available		✓
Interpretability required	✓	
Pattern recognition needed		✓

Data Categories

Structured vs. Unstructured Data

	Structured Data	Unstructured Data
Format	Tables, rows, columns	No fixed schema
Examples	Spreadsheets, SQL databases, CSV files	Images, text, audio, video
Processing	Traditional ML works well	Deep learning excels
Volume	Typically smaller	∼80% of all data generated
Feature extraction	Features are explicit (columns)	Features must be learned

Key Point

Deep learning's greatest advantage is its ability to process **unstructured data**—images, text, audio—without requiring manual feature engineering. This is why DL has revolutionized computer vision and NLP, where the input is inherently unstructured.

Learning Paradigms

Supervised Learning

The model learns from **labeled data**—input-output pairs (x_i, y_i). The goal is to learn a function f such that $f(x) \approx y$.

- **Classification**: output is a discrete category (e.g., spam vs. not spam, cat vs. dog)
- **Regression**: output is a continuous value (e.g., house price, temperature prediction)

Unsupervised Learning

The model learns from **unlabeled data**—it must discover patterns on its own.

- **Clustering**: grouping similar data points (e.g., customer segmentation)
- **Dimensionality reduction**: compressing data while preserving structure (e.g., PCA, t-SNE)
- **Generative models**: learning the underlying data distribution (e.g., GANs, VAEs)

Reinforcement Learning

An **agent** learns by interacting with an **environment**, receiving **rewards** or **penalties** for its actions. The goal is to learn a **policy** that maximizes cumulative reward.

- Examples: game playing (AlphaGo), robotics, autonomous driving
- No labeled data—the agent learns from trial and error

Self-Check Q1

A company wants to automatically organize customer support emails into topics without any predefined categories. Which learning paradigm should they use? Why? *(Solution in Appendix)*

Machine Learning vs. Deep Learning

Six Key Differences

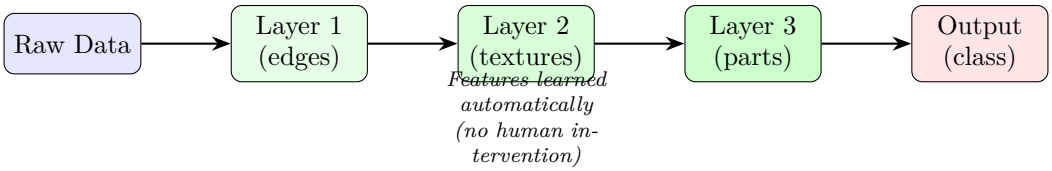
Aspect	Machine Learning	Deep Learning
Feature extraction	Manual (human-designed)	Automatic (learned from data)
Data dependency	Can work with small datasets	Requires large datasets
Computation	CPU is often sufficient	Requires GPU/TPU
Interpretability	Interpretable (white box)	Opaque (black box)
Versatility	Domain-specific features	Same architecture across domains
Performance vs. data	Plateaus with more data	Keeps improving with more data

The Feature Extraction Gap

This is the most important difference. In traditional ML:

1. A domain expert designs features (e.g., "count the number of edges," "compute the histogram of gradients")
2. These features are fed to a classifier (SVM, Random Forest, etc.)
3. If the features are poor, the model fails—no matter how good the classifier is

   In deep learning, the network learns its own features:



Performance Scaling

Traditional ML performance **plateaus** as data increases—the manually designed features become the bottleneck. Deep learning performance **keeps improving** with more data because it can learn increasingly rich representations. This is why DL dominates in the era of big data.

Computer Vision Foundations

How Computers See Images

An image is stored as a **matrix of numbers**. Each pixel has an intensity value in the range $[0, 255]$:

- **Grayscale image**: a single 2D matrix of size $H \times W$
- **Color image (RGB)**: three 2D matrices stacked—one for Red, one for Green, one for Blue

- A color image of size 224×224 has $224 \times 224 \times 3 = 150{,}528$ values

Intuition

When you see a photograph, your brain instantly recognizes objects. A computer sees only a grid of numbers. The challenge of computer vision is: **how do we go from a matrix of pixel values to semantic understanding** ("this is a cat")?

From Pixels to Features

Traditional computer vision used handcrafted feature descriptors:

- **SIFT** (Scale-Invariant Feature Transform): detects keypoints
- **HOG** (Histogram of Oriented Gradients): captures edge directions
- These features were then fed to classifiers like SVMs

The problem: designing good features required deep domain expertise and worked only for specific tasks. Deep learning changed this by learning features directly from data.

The ImageNet Challenge

What is ImageNet?

ImageNet is a large-scale visual recognition benchmark:

- ~1.2 million training images
- 1,000 object categories
- The **ImageNet Large Scale Visual Recognition Challenge (ILSVRC)** ran from 2010 to 2017

The Deep Learning Revolution

Year	Model	Top-5 Error	Significance
2010	NEC-UIUC	28.2%	Handcrafted features
2012	**AlexNet**	**16.4%**	First deep CNN to win; started the DL revolution
2014	GoogLeNet	6.7%	Inception modules
2014	VGG	7.3%	Deeper is better (19 layers)
2015	**ResNet**	**3.57%**	152 layers; surpassed human-level performance
—	Human	5.1%	Human-level benchmark

Turning Point: 2012

The **AlexNet moment** (Krizhevsky et al., 2012) was a watershed event. AlexNet reduced the error rate from 28.2% to 16.4%—a **40% relative improvement**—using a deep CNN trained on GPUs. This single result convinced the entire AI community that deep learning was viable, and every winner after 2012 used deep neural networks.

Self-Check Q2

In 2015, ResNet achieved a top-5 error of 3.57%, while human-level performance is approximately 5.1%. Does this mean ResNet "sees" better than humans? What does this comparison really tell us? *(Solution in Appendix)*

Convolutional Neural Networks (CNNs)

Core Idea

CNNs exploit the **spatial structure** of images by using **local filters** (kernels) that slide across the input, detecting patterns at every spatial position. Key properties:

- **Parameter sharing**: the same filter is applied across the entire image (far fewer parameters than a fully connected network)
- **Local connectivity**: each neuron connects only to a small region of the input
- **Translation invariance**: a cat is a cat regardless of where it appears in the image

Building Blocks of a CNN

1. Convolution Layer

A filter of size $k \times k$ slides across the input, computing element-wise multiplication and summation at each position:

$$\text{Output}(i,j) = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} \text{Input}(i+m,\ j+n) \cdot \text{Filter}(m,n) + b \tag{1}$$

Output size formula:

$$\boxed{O = \frac{W - K + 2P}{S} + 1} \tag{2}$$

where W = input size, K = kernel size, P = padding, S = stride.
Example: Input 32×32, kernel 5×5, padding = 0, stride = 1:

$$O = \frac{32 - 5 + 0}{1} + 1 = 28$$

2. Activation Function (ReLU)

After each convolution, a non-linearity is applied:

$$\text{ReLU}(x) = \max(0, x)$$

This introduces non-linearity, allowing the network to learn complex patterns. Negative values are set to zero.

3. Pooling Layer

Reduces the spatial dimensions while retaining the most important information:

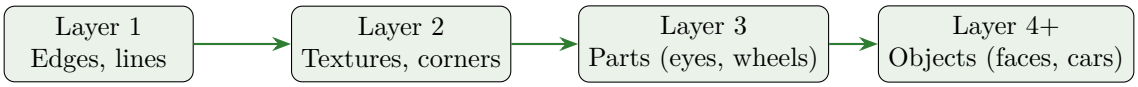
- **Max pooling**: takes the maximum value in each window (most common)
- **Average pooling**: takes the mean value in each window
- Typical: 2×2 window with stride 2, reducing each dimension by half

4. Fully Connected (FC) Layer

After several convolution + pooling stages, the feature maps are **flattened** into a 1D vector and fed to fully connected layers that perform classification.

Feature Learning Hierarchy

CNNs learn a hierarchy of increasingly abstract features:



Intuition

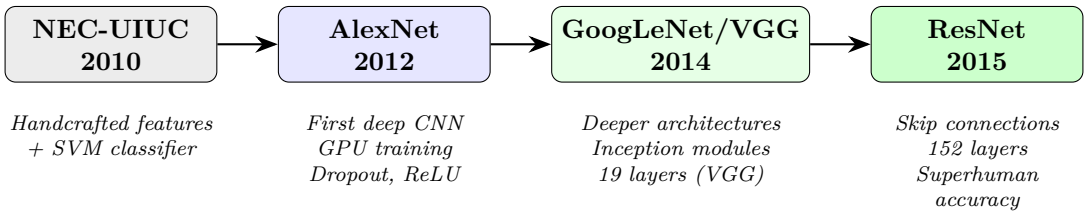
Think of each layer as asking increasingly sophisticated questions. Layer 1: "Are there edges here?" Layer 2: "Do edges form textures?" Layer 3: "Do textures form parts?" Layer 4: "Do parts form a recognizable object?" This compositionality is why deep networks are so powerful.

Self-Check Q3

Given an input image of size 64×64, a convolution layer with 32 filters of size 3×3, padding $= 1$, and stride $= 1$, what is the output size? How many learnable parameters does this layer have (including biases)? *(Solution in Appendix)*

Deep Learning Evolution: 2010–2015

From Handcrafted Features to Learned Representations



Key innovations at each stage:

- **AlexNet (2012)**: demonstrated that deep CNNs trained on GPUs with ReLU activation and dropout regularization dramatically outperform handcrafted features
- **VGG (2014)**: showed that using many small (3×3) filters is better than fewer large filters—deeper is better
- **GoogLeNet (2014)**: introduced Inception modules that apply multiple filter sizes in parallel, capturing multi-scale features efficiently
- **ResNet (2015)**: introduced **skip (residual) connections** that allow gradients to flow directly through the network, enabling training of extremely deep networks (152 layers)

Why ResNet Works

In very deep networks, adding more layers can actually **hurt** performance (degradation problem). ResNet solves this with skip connections: if a layer has nothing useful to learn, it can simply pass the input through unchanged ($\mathcal{F}(x) + x$). The network only needs to learn the **residual**—the difference from the identity mapping.

Computer Vision Tasks

The major computer vision tasks form a hierarchy of increasing complexity:

Task	Question Answered	Output
Classification	"What is in the image?"	Single label (e.g., "cat")
Localization	"Where is it?"	Label + bounding box
Object Detection	"Where are all objects?"	Multiple labels + bounding boxes
Instance Segmentation	"Which exact pixels belong to each object?"	Pixel-level mask per instance

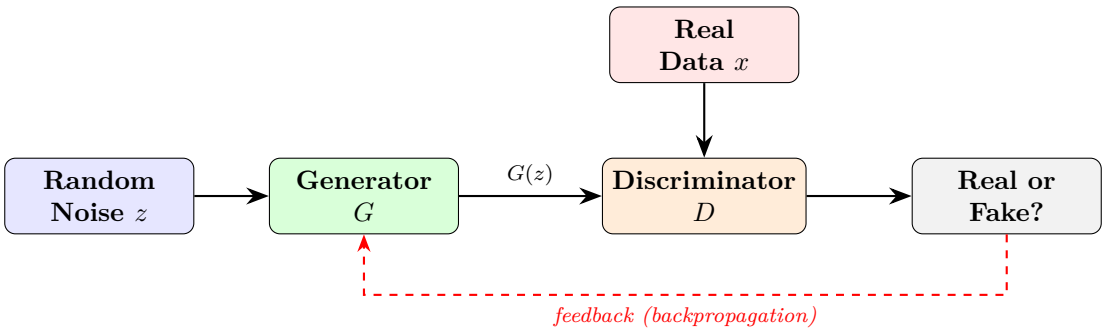
Intuition

Think of it as increasing levels of detail:
Classification: "There's a cat."
Localization: "There's a cat, and it's in the top-right corner."
Detection: "There are 2 cats and 1 dog, here's where each one is."
Segmentation: "Here are the exact pixel boundaries of each animal."

Generative Adversarial Networks (GANs)

The Core Idea

A GAN consists of two neural networks competing against each other:



- **Generator** (G): takes random noise z and produces fake data $G(z)$. Goal: fool the discriminator.
- **Discriminator** (D): takes an input (real or fake) and outputs a probability that it is real. Goal: correctly distinguish real from fake.

The Counterfeiter Analogy

The **Generator** is like a counterfeiter trying to produce fake money. The **Discriminator** is like a detective trying to catch counterfeit bills. As the detective improves, the counterfeiter must produce better fakes. As the counterfeiter improves, the detective must get better at spotting fakes. Both improve through this adversarial process until the fake money is indistinguishable from real money.

The GAN Loss Functions

The GAN training objective is a **minimax (zero-sum) game**:

$$\boxed{\min_G \max_D \; V(D,G) = \mathbb{E}_{x\sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z\sim p_z}[\log(1 - D(G(z)))]} \tag{3}$$

Breaking this down into each player's perspective:

Discriminator's objective (maximize):

$$\mathcal{L}_D = \mathbb{E}_{x\sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z\sim p_z}[\log(1 - D(G(z)))] \tag{4}$$

- $\log D(x)$: D wants $D(x) \to 1$ for real data (maximize $\log 1 = 0$)
- $\log(1 - D(G(z)))$: D wants $D(G(z)) \to 0$ for fake data (maximize $\log 1 = 0$)

Generator's objective (minimize):

$$\mathcal{L}_G = \mathbb{E}_{z\sim p_z}[\log(1 - D(G(z)))] \tag{5}$$

- G wants $D(G(z)) \to 1$, so $\log(1-1) = \log 0 \to -\infty$ (minimized)
- In practice, G maximizes $\log D(G(z))$ instead (better gradient signal early in training)

Training Process

GAN training alternates between two steps:

1. **Train** D: Fix G, update D to better classify real vs. fake
2. **Train** G: Fix D, update G to better fool D

   At equilibrium: $D(x) = 0.5$ for all inputs—the discriminator cannot tell real from fake.

Applications

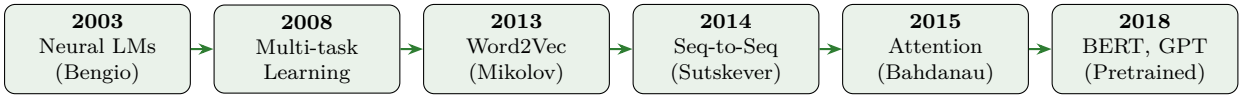
- **Image synthesis**: generating photorealistic faces (StyleGAN)
- **Image-to-image translation**: converting sketches to photos, day to night
- **Super-resolution**: enhancing low-resolution images
- **Data augmentation**: generating synthetic training data
- **Domain adaptation**: adapting models from one domain (e.g., synthetic data) to another (e.g., real-world data)

Self-Check Q4

In a GAN, what happens if the discriminator becomes too strong too quickly? Why is balancing G and D training important? *(Solution in Appendix)*

NLP Evolution Timeline

Natural Language Processing has undergone a series of paradigm shifts:



- **2003 – Neural Language Models**: Bengio et al. proposed learning word representations as part of a language model, replacing n-gram count-based methods
- **2008 – Multi-task Learning**: showed that a single neural network could learn multiple NLP tasks simultaneously, learning shared representations
- **2013 – Word2Vec**: Mikolov et al. introduced efficient training of word embeddings; demonstrated that word vectors capture semantic relationships (e.g., king − man + woman ≈ queen)
- **2014 – Sequence-to-Sequence**: Sutskever et al. introduced encoder-decoder architecture for machine translation, processing entire sentences as sequences
- **2015 – Attention Mechanism**: Bahdanau et al. allowed models to focus on relevant parts of the input, solving the information bottleneck of fixed-length encoding
- **2018 – Pretrained Models**: BERT (Google) and GPT (OpenAI) showed that pretraining on massive text corpora and fine-tuning on specific tasks achieves state-of-the-art results across NLP

The Big Shift

The evolution from Word2Vec (2013) to BERT/GPT (2018) represents a shift from **static word embeddings** (same vector for "bank" in all contexts) to **contextual embeddings** (different vector depending on surrounding words). This is the central theme of this course.

NLP Models: RNN, LSTM, GRU

Recurrent Neural Networks (RNN)

RNNs process sequences **one element at a time**, maintaining a hidden state that serves as memory:

$$h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t + b) \tag{6}$$

Strengths: natural fit for sequential data, parameter sharing across time steps.
Weakness: **vanishing gradient problem**—when sequences are long, gradients shrink exponentially during backpropagation through time, making it nearly impossible to learn long-range dependencies.

Why Gradients Vanish

During backpropagation, the gradient is multiplied by W_h at each time step. If the eigenvalues of W_h are < 1, repeated multiplication drives the gradient to zero. For a sequence of length 100: $0.9^{100} \approx 0.00003$.

Long Short-Term Memory (LSTM)

LSTM solves the vanishing gradient problem by introducing a **cell state** C_t and three **gates**:

- **Forget gate**: $f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$ — what to discard
- **Input gate**: $i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$ — what to store
- **Output gate**: $o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$ — what to output
- **Cell update**: $C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t$ (additive update preserves gradients)

Gated Recurrent Unit (GRU)

GRU simplifies LSTM by merging the cell state and hidden state, using only two gates:

- **Reset gate**: $r_t = \sigma(W_r[h_{t-1}, x_t] + b_r)$ — how much past to forget
- **Update gate**: $z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$ — keep old vs. use new
- **Hidden state**: $h_t = (1 - z_t) \cdot h_{t-1} + z_t \cdot \tilde{h}_t$ (interpolation)

  GRU has fewer parameters, trains faster, and performs comparably to LSTM on many tasks.

NLP Applications

Deep learning has enabled breakthroughs across a wide range of NLP tasks:

Application	Description	Example
Text Classification	Assign categories to text	Spam detection, sentiment analysis
Information Extraction	Extract structured data from text	Named Entity Recognition (NER)
Conversational Agents	Generate contextual responses	Chatbots, virtual assistants
Information Retrieval	Find relevant documents	Search engines
Question Answering	Answer questions from context	SQuAD, reading comprehension
Machine Translation	Translate between languages	Google Translate
Text Summarization	Compress text to key points	News summaries

Industry Impact

NLP is transforming industries: healthcare (clinical note analysis), finance (sentiment-driven trading), legal (contract analysis), customer service (automated support), and education (automated grading). The common thread is that these applications require understanding **unstructured text data**, which is where deep learning excels.

Self-Check Q5

A hospital wants to automatically extract drug names, dosages, and conditions from doctors' handwritten clinical notes (after OCR). Which NLP task is this? Which learning paradigm (supervised, unsupervised, or reinforcement) would you use? *(Solution in Appendix)*

Self-Attention and Transformers Preview

The Limitation of RNNs

RNNs process tokens **sequentially**: token t must wait for tokens $1, 2, \ldots, t-1$ to be processed. This creates two problems:

1. **No parallelization**: training is slow because computations cannot be distributed across GPU cores
2. **Information bottleneck**: the entire input sequence is compressed into a single fixed-size vector h_T

The Self-Attention Innovation

Self-attention allows every token to **directly attend to every other token** in the sequence, regardless of distance:

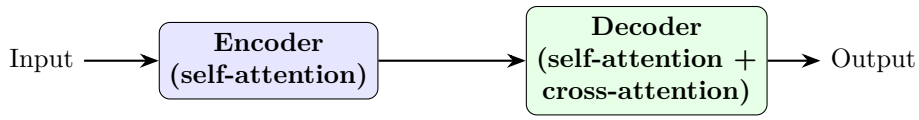
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \tag{7}$$

where Q (queries), K (keys), and V (values) are linear projections of the input. The key properties:

- **Parallel computation**: all attention weights are computed simultaneously via matrix multiplication
- **Direct long-range connections**: token 1 can directly attend to token 100 (no gradient chain)
- **Learned relevance**: the model learns which tokens are most relevant to each other through attention weights

Transformer Architecture (Preview)

The Transformer (Vaswani et al., 2017) uses self-attention as its core mechanism:



- **Encoder**: processes the input sequence, producing contextual representations
- **Decoder**: generates the output sequence, attending to both its own previous outputs and the encoder's representations
- Both use **multi-head attention**: multiple attention operations in parallel, each learning different aspects of the relationships between tokens

Transformers will be covered in depth in later lectures.

Self-Check Q6

Why is the scaling factor $\sqrt{d_k}$ needed in the attention formula? What would happen without it? *(Solution in Appendix)*

Sequential Data and Model Architectures

Types of Sequential Data

Sequential data appears in many domains:

- **Text**: sequences of words or characters
- **Speech**: sequences of audio frames
- **Time series**: stock prices, sensor readings, weather data
- **Video**: sequences of image frames
- **DNA**: sequences of nucleotides (A, C, G, T)

Sequence-to-Sequence Architectures

Different tasks require different input-output configurations:

Architecture	Input → Output	Example Task	Concrete Example
One-to-One	Fixed → Fixed	Image classification	Image → "cat"
One-to-Many	Fixed → Sequence	Image captioning	Image → "A cat sitting on a mat"
Many-to-One	Sequence → Fixed	Sentiment analysis	"I love this movie" → Positive
Many-to-Many (equal)	Sequence → Sequence	Named entity recognition	Word-by-word tagging
Many-to-Many (unequal)	Sequence → Sequence	Machine translation	English sentence → Arabic sentence

Intuition

The flexibility of sequence models is what makes them so powerful. The **same underlying architecture** (RNN/LSTM/Transformer) can be configured for any of these patterns simply by changing what you feed in and what you read out.

Origin of Recurrent Neural Networks

Elman 1990: "Finding Structure in Time"

The concept of recurrent neural networks was introduced by **Jeffrey Elman** in his seminal 1990 paper *"Finding Structure in Time."*

Key ideas:

- Traditional neural networks treat inputs as independent—they have no notion of **temporal order**
- Elman proposed adding a **context layer** (hidden state) that stores information from previous time steps
- At each time step, the network receives both the current input **and** a copy of its own previous hidden state
- This recurrence gives the network **memory**—the ability to process sequences where order matters

The Elman Network:

$$h_t = f(W_x \cdot x_t + W_h \cdot h_{t-1}) \tag{8}$$

This is the same equation used in modern RNNs. The context units act as a **short-term memory**, allowing the network to learn temporal patterns.

Historical Significance

Elman's 1990 paper demonstrated that simple recurrent networks could learn grammatical structure from sequences of words without being explicitly taught grammar rules. This was a foundational insight: **structure in language can emerge from processing sequences with memory**. This idea directly led to the RNN, LSTM, GRU, and ultimately the Transformer architectures used today.

Self-Check Q7

Elman's 1990 network and a modern vanilla RNN use essentially the same equation. What key advances between 1990 and 2012 made RNNs practical for real-world applications? Name at least three. *(Solution in Appendix)*

Appendix: Solutions to Self-Check Questions

Q1: Which learning paradigm for organizing emails without predefined categories?

Unsupervised learning, specifically **clustering**. Since there are no predefined categories, we cannot use supervised learning (which requires labeled data). Clustering algorithms (e.g., k-means, hierarchical clustering, or topic modeling with LDA) can automatically discover groups of similar emails based on their content. The company can then inspect each cluster to assign meaningful topic labels after the fact. If the company later wants to classify new emails into the discovered categories, they can switch to supervised learning using the clusters as training labels.

Q2: Does ResNet "see" better than humans?

Not exactly. The comparison is misleading for several reasons:

- The 5.1% human error rate was measured on a specific person (Andrej Karpathy) classifying ImageNet images into 1,000 fine-grained categories—many of which are extremely similar dog breeds that require expert knowledge
- ResNet excels at this **narrow task** (classifying among 1,000 specific categories with clean, centered images)
- Humans vastly outperform ResNet in: understanding context, recognizing objects in unusual orientations, generalizing from few examples, handling adversarial perturbations, and performing open-ended visual reasoning
- The comparison tells us that deep learning has **surpassed human-level on a specific benchmark**, not that it has achieved general visual intelligence

Q3: CNN output size and parameter count

Output spatial size:

$$O = \frac{W - K + 2P}{S} + 1 = \frac{64 - 3 + 2(1)}{1} + 1 = 64$$

With 32 filters, the output is $64 \times 64 \times 32$.

Learnable parameters: Each filter has size 3×3 and operates on the input (assuming 3 RGB channels): $3 \times 3 \times 3 = 27$ weights per filter, plus 1 bias $= 28$ parameters per filter. With 32 filters: $32 \times 28 =$ **896** parameters.

Note: if the input has only 1 channel (grayscale), the count would be $32 \times (3 \times 3 \times 1 + 1) = 320$.

Q4: What if the discriminator becomes too strong?

If the discriminator becomes too strong too quickly, it will correctly reject all generated samples with high confidence ($D(G(z)) \approx 0$). This causes the generator's gradient to **vanish**:

- The generator's loss involves $\log(1 - D(G(z)))$. When $D(G(z)) \approx 0$, this becomes $\log(1) = 0$, providing **no useful gradient signal** for the generator to improve
- The generator cannot learn because it receives no information about *how* to improve its outputs

- This leads to **mode collapse**: the generator produces limited, repetitive outputs

Balancing is crucial because both networks must improve at roughly the same pace. In practice, the discriminator is often trained for fewer steps per iteration, or techniques like label smoothing and gradient penalty are used to prevent it from becoming too confident.

Q5: NLP task for extracting medical information

This is **Information Extraction**, specifically **Named Entity Recognition (NER)**. The task involves identifying and classifying specific entities in text:

- Drug names → DRUG entity
- Dosages → DOSAGE entity
- Medical conditions → CONDITION entity

This requires **supervised learning**: we need a labeled training dataset where medical professionals have annotated clinical notes with the correct entity types. The model (typically a BiLSTM-CRF or fine-tuned BERT) learns to predict entity labels for each token. This is a **many-to-many** sequence labeling task.

Q6: Why scale by $\sqrt{d_k}$ in attention?

The dot product QK^T computes the sum of d_k products. As d_k grows, the magnitude of the dot products grows proportionally (assuming unit-variance inputs, the variance of the dot product is d_k). Large dot product values push the softmax function into regions with **extremely small gradients** (saturation):

- If $d_k = 512$, dot products can be in the range $[-30, +30]$
- $\text{softmax}([30, -30]) \approx [1.0, 0.0]$—essentially a hard, one-hot distribution
- Gradients of softmax at saturation are near zero, causing **vanishing gradients**

Dividing by $\sqrt{d_k}$ normalizes the dot products to have unit variance regardless of d_k, keeping them in a range where softmax produces **smooth probability distributions** with healthy gradients.

Q7: What advances made RNNs practical between 1990 and 2012?

1. **LSTM (Hochreiter & Schmidhuber, 1997)**: solved the vanishing gradient problem that made vanilla RNNs unable to learn long-range dependencies—this was the most critical theoretical advance
2. **GPU computing**: massively parallel hardware (especially NVIDIA GPUs with CUDA) made training large networks on large datasets feasible. Training that took weeks on CPUs could be done in hours on GPUs
3. **Large datasets**: the internet enabled collection of massive text and speech corpora needed for training data-hungry deep models
4. **Better optimization**: advances in optimization algorithms (Adam, RMSProp), initialization strategies (Xavier/He initialization), and regularization techniques (dropout, batch normalization) made training deep networks more stable
5. **Software frameworks**: libraries like Theano (2010), later TensorFlow (2015) and PyTorch (2016), made it dramatically easier to implement, train, and debug neural networks with automatic differentiation